

Title:

Implement a Neural Network Trained with Mini-Batch Gradient Descent and Momentum (From Scratch)

Objective:

You will implement a **two-layer neural network** (1 hidden layer + 1 output layer) **from scratch** in **pure Python** (no NumPy, no frameworks) and train it using **Mini-Batch Gradient Descent with Momentum** to learn the **XOR** classification problem.

Background

In previous exercises, you may have:

- Implemented basic **backpropagation** with:
 - Stochastic Gradient Descent (SGD): update after every sample
 - Mini-Batch Gradient Descent: update after a small batch of samples

In this assignment, you must **extend mini-batch training** by adding **momentum**, which helps:

- Smooth out noisy gradients
 - Speed up learning along consistent directions
 - Reduce zig-zagging in narrow valleys
-

Requirements

1. Network Architecture

- Inputs: 2 features (for XOR)
- Hidden layer: 2 or more neurons
- Output layer: 2 neurons (class 0 and class 1, using one-hot encoding)
- Activation function: **sigmoid** for all neurons

2. Dataset (XOR)

Use this dataset:

x1	x2	label
0	0	0
0	1	1
1	0	1
1	1	0

3. Training Algorithm: Mini-Batch + Momentum

- Use a **mini-batch size** (e.g., 2).
- For each batch:
 1. For each sample:

- Forward pass → outputs
 - Backward pass → deltas
 - Accumulate gradients in a buffer (don't update weights yet)
2. After processing the entire batch:
- Compute the **average gradient** per weight
 - Use **momentum** update:

$$v_{ij} \leftarrow \mu \cdot v_{ij} - \eta \cdot g_{ij}$$

$$w_{ij} \leftarrow w_{ij} + v_{ij}$$

Where:

- v_{ij} = velocity for weight w_{ij}
 - μ = momentum factor (e.g., 0.9)
 - η = learning rate
 - g_{ij} = average gradient of loss w.r.t. w_{ij} over the batch
- You must:
 - Maintain a **velocity vector** for each neuron's weights
 - Initialize all velocities to zero at the start of training

4. Outputs to Show

- During training: print error (sum of squared errors) every few epochs
- After training:
 - Print predictions for all 4 XOR inputs
 - Show the predicted class for each

5. Goal of the Assignment

By completing this assignment, you should:

- Understand how **momentum** modifies the basic gradient descent update
- See how momentum + mini-batch can make learning **more stable and faster**
- Gain deeper intuition by implementing everything **from first principles** (no libraries)